

Практическое занятие 6.1

Двухклассовая классификация: сигмоида, функции потерь и градиентный спуск

Дисциплина: Машинное обучение и анализ данных

1) Цели занятия

После выполнения ПЗ-6.1 студент должен:

1. Понимать, как строится линейная модель для классификации и как из линейного «скора» получается вероятность класса.
2. Уметь реализовать и объяснить две функции потерь для двухклассовой классификации:
 - Sigmoidal Least Squares (квадратичная ошибка на вероятности),
 - Cross-Entropy (кросс-энтропия / логистическая потеря).
3. Уметь вручную реализовать градиенты этих потерь (без autograd).
4. Уметь реализовать градиентный спуск (GD), управляя гиперпараметрами обучения.
5. Уметь оценивать качество модели:
 - accuracy,
 - confusion matrix (TN, FP, FN, TP),
 - (дополнительно) precision/recall,и делать корректные выводы.
6. Уметь проводить простые исследовательские эксперименты (изменение learning rate, числа итераций, порога классификации и т.п.) и оформлять результаты как мини-исследование.

2) Что именно делаем на ПЗ-6.1 (содержание)

ПЗ-6.1 — это «фундамент» материала лекции 8 (**функции потерь + обучение градиентным спуском** на задаче двухклассовой классификации).

Мы последовательно выполняем:

1. загрузку и подготовку данных (без скачивания из интернета);
2. построение модели $a(x) = w_0 + w^\top x$ и вероятности $\hat{y} = \sigma(a(x))$;

3. реализацию потерь LS и CE;
4. вывод/реализацию градиентов;
5. реализацию GD;
6. обучение модели и сравнение CE vs LS по:
 - графикам потерь,
 - качеству на тесте,
 - матрице ошибок.

3) Теория, которую студент обязан понимать и отразить в отчете

3.1. Линейная часть модели

$$a(x) = w_0 + w^\top x$$

Это значение может быть любым (от $-\infty$ до $+\infty$).

3.2. Сигмоида и вероятность

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \hat{y} = \sigma(a(x))$$

Интерпретация: $\hat{y}$ — **оценка вероятности** класса 1.

3.3. Две функции потерь

(A) Sigmoidal Least Squares (LS)

$$J_{LS}(w) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Смысл: «приближаем вероятность к метке», но для классификации это не всегда оптимально.

(B) Cross-Entropy (CE)

$$J_{CE}(w) = -\frac{1}{n} \sum_{i=1}^n (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

Это стандарт для логистической регрессии; обычно дает более корректную геометрию оптимизации.

3.4. Градиентный спуск

$$w \leftarrow w - \eta \nabla J(w)$$

η — шаг обучения.

3.5. Важнейший результат для CE

Для CE + сигмоиды градиент получается особенно простой:

$$\nabla J_{CE}(w) = \frac{1}{n} X_b^\top (\hat{y} - y)$$

где X_b — матрица признаков с добавленным столбцом единиц (bias).

Для LS градиент содержит дополнительный множитель $\hat{y}(1 - \hat{y})$, что часто ухудшает обучение при насыщении сигмоиды.

4) План занятия

Часть 1

Подготовка данных: загрузка, train/test split, стандартизация.

Часть 2

Реализация модели: bias, sigmoid, predict_proba.

Часть 3

Реализация функций потерь LS и CE + численная проверка адекватности (значения, стабильность log).

Часть 4

Градиенты + GD + обучение CE vs LS + метрики качества.

Домашнее задание

Индивидуальные исследования по вариантам + оформление отчета.

5) Пошаговая инструкция студенту (что делать в ноутбуке)

Студент открывает ноутбук `PZ_6_1_Training_and_Losses_RU.ipynb` и выполняет последовательно все ячейки.

Шаг 1. Импорты и фиксирование seed

Цель: воспроизводимость эксперимента.

Шаг 2. Загрузка данных breast_cancer

Цель: получить (X) и (y) без внешних файлов и интернет-скачиваний.

Шаг 3. Train/test split + стандартизация

Студент должен:

- объяснить, что такое утечка данных (data leakage) и почему scaler обучаем только на train.

Шаг 4. Реализация sigmoid, add_bias, predict_proba

Студент должен понимать:

- зачем нужен bias;
- почему делаем clip в sigmoid.

Шаг 5. Реализация loss_ls и loss_ce

Студент должен:

- пояснить смысл CE;
- объяснить, зачем epsilon/clip перед log.

Шаг 6. Реализация grad_ce и grad_ls

Студент должен:

- уметь объяснить разницу градиентов;
- проверить размерности градиента.

Шаг 7. Реализация gradient_descent

Студент должен:

- уметь менять lr, n_iters, lr_decay;
- понимать, что такое «расходимость обучения».

Шаг 8. Обучение модели CE vs LS

Студент строит графики loss и делает вывод:

- какая потеря сходится быстрее/стабильнее.

Шаг 9. Метрики и матрица ошибок

Студент должен:

- посчитать accuracy,
- confusion matrix (TN, FP, FN, TP),
- (дополнительно) precision/recall,
- интерпретировать, какие ошибки критичнее (FP или FN) в прикладном контексте.

6) Индивидуальные задания (15 вариантов)

Каждому студенту: **2 исследовательских задания + 2 контрольных вопроса.**

Вариант 1

Исследования: (1) CE: $lr = 0.05/0.1/0.2$ (графики loss). (2) CE: threshold = $0.3/0.5/0.7$ (confusion).

Вопросы: (1) Почему порог влияет на FP/FN? (2) Почему стандартизация ускоряет GD?

Вариант 2

Исследования: (1) LS: $lr = 0.1/0.3/0.5$. (2) LS: $lr_decay = 0/1e-4/1e-3$.

Вопросы: (1) Чем CE лучше согласуется с вероятностной моделью? (2) Что такое «насыщение сигмоиды»?

Вариант 3

Исследования: (1) CE: $n_iters = 500/1500/3000$. (2) accuracy(threshold) для $0.1 \dots 0.9$ (шаг 0.1).

Вопросы: (1) Почему больше итераций $\neq$ лучше на тесте? (2) Как понять по loss, что «сошлось»?

Вариант 4

Исследования: (1) Убрать стандартизацию и обучить CE при $lr=0.2$ (описать эффект). (2) Гистограммы predict_proba по классам.

Вопросы: (1) Как масштаб признаков меняет поверхность потерь? (2) Чем вероятность отличается от «уверенности»?

Вариант 5

Исследования: (1) 3 разных random_state разбиения (сравнить метрики). (2) test_size = 0.2 и 0.4.

Вопросы: (1) Зачем фиксировать seed? (2) Зачем stratify?

Вариант 6

Исследования: (1) CE: $lr=0.2$ без lr_decay и с $lr_decay=1e-4$ (сравнить). (2) CE: сравнить threshold 0.4 и 0.6.

Вопросы: (1) Зачем делают затухание шага? (2) Почему при изменении threshold accuracy может почти не меняться, а confusion меняется?

Вариант 7

Исследования: (1) LS: сравнить n_iters 1000 и 3000 при фиксированном lr .
(2) LS: подобрать lr , при котором обучение расходится (показать).

Вопросы: (1) Что означает «расходимость» GD? (2) Почему LS может быть менее стабильной на сигмоиде?

Вариант 8

Исследования: (1) CE: изменить начальные веса ($w=0$ и $w \sim N(0, 0.01)$) и сравнить сходимость. (2) CE: сравнить $lr=0.05$ и $lr=0.3$.

Вопросы: (1) Зависит ли минимум от инициализации? (2) Почему слишком большой lr ухудшает обучение?

Вариант 9

Исследования: (1) CE: построить precision/recall при $threshold=0.3/0.5/0.7$. (2) Выбрать $threshold$, где recall максимален (обосновать).

Вопросы: (1) Чем precision отличается от recall? (2) В каких задачах важнее recall?

Вариант 10

Исследования: (1) CE: сравнить обучение на полном наборе признаков и на первых 5 признаках. (2) Для обоих случаев сравнить loss и accuracy.

Вопросы: (1) Почему уменьшение числа признаков может ухудшить качество? (2) Почему иногда уменьшение признаков улучшает обобщение?

Вариант 11

Исследования: (1) CE: сравнить обучение на 60% и 90% данных (изменив $test_size$). (2) Сравнить confusion matrix.

Вопросы: (1) Как размер train влияет на качество? (2) Почему результаты на маленьких train более нестабильны?

Вариант 12

Исследования: (1) CE: подобрать lr так, чтобы loss падал максимально быстро, но не расходился. (2) Зафиксировать найденный lr и сравнить с lr в ноутбук.

Вопросы: (1) Как practically подбирать lr ? (2) Почему «самый большой lr » часто плох?

Вариант 13

Исследования: (1) LS: сравнить результат при threshold 0.5 и 0.65. (2) Сравнить precision/recall.

Вопросы: (1) Почему оптимизация LS влияет на распределение вероятностей? (2) Почему threshold — это отдельный слой решения поверх модели?

Вариант 14

Исследования: (1) SE: сделать 5 запусков с разными seed и оценить среднюю accuracy и разброс. (2) Сделать вывод о стабильности.

Вопросы: (1) Почему нужна оценка разброса? (2) Что такое воспроизводимость эксперимента в ML?

Вариант 15

Исследования: (1) SE: сравнить качество без стандартизации при очень маленьком lr (например $1e-4$) и со стандартизацией при $lr=0.2$. (2) Сделать вывод, что важнее и почему.

Вопросы: (1) Можно ли «компенсировать» отсутствие стандартизации очень маленьким lr ? (2) Почему это не лучший путь?

7) Требования к отчету студента

Отчет должен содержать:

1. Титульный блок (ФИО, группа, вариант, дата).
2. Цель работы (2–4 предложения).
3. Описание данных:
 - размерности,
 - распределение классов,
 - зачем train/test split,
 - зачем стандартизация.
4. Модель:
 - формулы $a(x)$, $\sigma(z)$, $\hat{y}$.
5. Потери:

- формулы LS и CE,
- объяснение смысла,
- объяснение epsilon/clip.

6. Обучение:

- формула GD,
- какие гиперпараметры использовались.

7. Эксперименты по варианту:

- настройки,
- графики loss,
- метрики (accuracy + confusion),
- интерпретация.

8. Ответы на 2 контрольных вопроса.

9. Итоговые выводы (не менее 10–12 предложений).

8) Что именно сдаёт студент

1. **ipynb-файл** с выполненными ячейками и графиками (по варианту).
2. **PDF/Word-отчет** (структура выше).


```
{
  "cells": [
    {
      "cell_type": "markdown",
      "id": "3e682c5c",
      "metadata": {},
      "source": [
        "# Практическое занятие 6.1 \n",
        "## Двухклассовая классификация: сигмоида, функции потерь и градиентный\nспуск\n",
        "\n",
        "**Дисциплина:** Машинное обучение и анализ данных \n",
        "**Формат:** Jupyter Notebook (Python) \n",
        "**Цель занятия:** научиться реализовывать «с нуля» ключевые функции\nпотерь для двухклассовой классификации и обучать модель градиентным спуском,\nanализируя качество и поведение обучения.\n",
        "\n",
        "---\n",
        "\n",
        "### Что студент должен уметь после выполнения\n",
        "1. Объяснить, что такое *линейная модель* и *сигмоидальная\nнелинейность*.\n",
        "2. Реализовать функции потерь:\n",
        "   - сигмоидальная **Least Squares** (квадратичная ошибка),\n",
        "   - **Cross-Entropy** (кросс-энтропия / логистическая потеря).\n",
        "3. Вывести и реализовать градиенты (вручную, без autograd).\n",
        "4. Реализовать градиентный спуск и управлять гиперпараметрами обучения\n(шаг, число итераций).\n",
        "5. Оценивать качество: accuracy, confusion matrix, и анализировать\nошибки.\n",
        "6. Делать корректные выводы по графикам обучения (loss и\niterations).\n",
        "\n",
        "---\n",
        "\n",
        "### Важное замечание\n",
        "Обычно используются внешние датасеты и библиотека `autograd`. \n",
        "В этой учебной версии мы делаем *полностью самодостаточный ноутбук*,\nбез скачивания данных из интернета:\n",
        "- для реального примера используем встроенный датасет `breast_cancer` из\n`scikit-learn`;\n",
        "- все градиенты считаем вручную.\n",
        "\n",
        "Если у вас нет `scikit-learn`, установите: `pip install scikit-learn`."
      ]
    },
    {
      "cell_type": "code",
      "execution_count": null,
      "id": "52ddea3b",
    }
  ],
}
```

```

"metadata": {},
"outputs": [],
"source": [
    "# Блок 1. Импорты и настройки\n",
    "\n",
    "import numpy as np\n",
    "import matplotlib.pyplot as plt\n",
    "\n",
    "from sklearn.datasets import load_breast_cancer\n",
    "from sklearn.model_selection import train_test_split\n",
    "from sklearn.preprocessing import StandardScaler\n",
    "\n",
    "# Для воспроизводимости\n",
    "np.random.seed(42)"
]
},
{
    "cell_type": "markdown",
    "id": "b3dace38",
    "metadata": {},
    "source": [
        "## Блок 2. Загрузка данных (breast cancer) и подготовка\n",
        "\n",
        "**Задача:** двухклассовая классификация. \n",
        "Целевая переменная:  $y \in \{0, 1\}$  (в датасете: 0 – malignant, 1 – benign).\n",
        "\n",
        "### Почему нужна стандартизация признаков?\n",
        "Градиентный спуск чувствителен к масштабу признаков. \n",
        "Если один признак меняется в диапазоне 0...1, а другой 0...1000, то:\n",
        "- поверхность функции потерь становится «вытянутой»,\n",
        "- обучение будет либо очень медленным, либо нестабильным.\n",
        "\n",
        "Поэтому мы используем StandardScaler: вычитаем среднее и делим на стандартное отклонение."
    ]
}
],
{
    "cell_type": "code",
    "execution_count": null,
    "id": "891a4e3a",
    "metadata": {},
    "outputs": [],
    "source": [
        "# Загрузка\n",
        "data = load_breast_cancer()\n",
        "X = data.data          # матрица признаков (n_samples, n_features)\n",
        "y = data.target        # метки классов (n_samples,)\n",
        "\n",
        "print(\"X shape:\", X.shape)\n",

```

```

    "print(\"y shape:\", y.shape)\n",
    "print(\"Классы y:\", np.unique(y, return_counts=True))"
]
},
{
    "cell_type": "code",
    "execution_count": null,
    "id": "673dc9e4",
    "metadata": {},
    "outputs": [],
    "source": [
        "# Деление на обучающую и тестовую выборки\n",
        "X_train, X_test, y_train, y_test = train_test_split(\n",
        "    X, y, test_size=0.25, random_state=42, stratify=y\n",
        ")\n",
        "\n",
        "# Стандартизация по обучающей выборке\n",
        "scaler = StandardScaler()\n",
        "X_train_sc = scaler.fit_transform(X_train)\n",
        "X_test_sc = scaler.transform(X_test)\n",
        "\n",
        "print(\"Train:\", X_train_sc.shape, \"Test:\", X_test_sc.shape)"
    ]
},
{
    "cell_type": "markdown",
    "id": "a8a94491",
    "metadata": {},
    "source": [
        "## Блок 3. Линейная модель и сигмоида\n",
        "\n",
        "Мы используем линейную модель вида:\n",
        "\n",

$$a(x) = w_0 + w^{\text{top}} x$$

        "\n",
        "Чтобы превратить скор в вероятность класса 1, применяем  

        **сигмоиду**:\n",
        "\n",

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

        "\n",
        "Тогда прогноз вероятности:\n",

$$\hat{y} = \sigma(a(x)).$$

    ]
}
]

```

```

},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "e0c828eb",
  "metadata": {},
  "outputs": [],
  "source": [
    "def add_bias(X: np.ndarray) -> np.ndarray:\n",
    "    \"\"\"Добавляет столбец единиц для учета смещения w0.\"\"\"\n",
    "    return np.hstack([np.ones((X.shape[0], 1)), X])\n",
    "\n",
    "def sigmoid(z: np.ndarray) -> np.ndarray:\n",
    "    \"\"\"Сигмоида с защитой от переполнения.\"\"\"\n",
    "    z = np.clip(z, -50, 50)\n",
    "    return 1.0 / (1.0 + np.exp(-z))\n",
    "\n",
    "# Проверка сигмоиды на нескольких значениях\n",
    "test_z = np.array([-10, -2, 0, 2, 10], dtype=float)\n",
    "print(\"z:\", test_z)\n",
    "print(\"sigmoid(z):\", sigmoid(test_z))"
  ]
},
{
  "cell_type": "markdown",
  "id": "5c3cceeб",
  "metadata": {},
  "source": [
    "## Блок 4. Функции потерь: Least Squares и Cross-Entropy\n",
    "\n",
    "\n",

$$J_{LS}(w) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

    \n",
    "\n",

$$J_{CE}(w) = -\frac{1}{n} \sum_{i=1}^n \left( y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i) \right)$$

    \n",
    "\n",
    "Для CE добавляем маленькое  $\epsilon$ , чтобы избежать  $\log(0)$ ."
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "639617d2",
  "metadata": {},
  "outputs": [],
  "source": [

```

```

def predict_proba(Xb: np.ndarray, w: np.ndarray) -> np.ndarray:\n",
    \"\"\"Предсказание вероятностей для класса 1.\"\"\"\n",
    return sigmoid(Xb @ w)\n",
\n",
def loss_ls(Xb: np.ndarray, y: np.ndarray, w: np.ndarray) -> float:\n",
    \"\"\"Least Squares для сигмоидальной модели.\"\"\"\n",
    y_hat = predict_proba(Xb, w)\n",
    return float(np.mean((y_hat - y) ** 2))\n",
\n",
def loss_ce(Xb: np.ndarray, y: np.ndarray, w: np.ndarray, eps: float =
1e-12) -> float:\n",
    \"\"\"Cross-Entropy (логистическая потеря).\"\"\"\n",
    y_hat = predict_proba(Xb, w)\n",
    y_hat = np.clip(y_hat, eps, 1 - eps)\n",
    return float(-np.mean(y * np.log(y_hat) + (1 - y) * np.log(1 -
y_hat)))\n",
\n",
Xb_train = add_bias(X_train_sc)\n",
w0 = np.zeros(Xb_train.shape[1])\n",
\n",
print(\"LS loss (w=0):\", loss_ls(Xb_train, y_train, w0))\n",
print(\"CE loss (w=0):\", loss_ce(Xb_train, y_train, w0))
]
},
{
    "cell_type": "markdown",
    "id": "c2c4e169",
    "metadata": {},
    "source": [
        \"\"\" Блок 5. Градиенты (реализация вручную)\n\",
        \n\",
        \"Для Cross-Entropy + сигмоида:\n\",
        \n\",
        \"\n\",
        \"\\nabla J_{CE}(w) = \\frac{1}{n} X_b^{\\top} (\\hat{y} - y)\n\",
        \"\n\",
        \"Для Least Squares + сигмоида:\n\",
        \n\",
        \"\n\",
        \"\\nabla J_{LS}(w) = \\frac{2}{n} X_b^{\\top} \\left( (\\hat{y} - y) \\cdot \\hat{y} (1 - \\hat{y}) \\right)\n\",
        \"\n\"
    ]
},
{
    "cell_type": "code",
    "execution_count": null,
    "id": "3323b59d",
    "metadata": {},

```

```

"outputs": [],
"source": [
    "def grad_ce(Xb: np.ndarray, y: np.ndarray, w: np.ndarray) ->
np.ndarray:\n",
    "    \"\"\"Градиент Cross-Entropy для сигмоидальной модели.\"\"\"\n",
    "    y_hat = predict_proba(Xb, w)\n",
    "    n = Xb.shape[0]\n",
    "    return (Xb.T @ (y_hat - y)) / n\n",
    "\n",
    "def grad_ls(Xb: np.ndarray, y: np.ndarray, w: np.ndarray) ->
np.ndarray:\n",
    "    \"\"\"Градиент Least Squares для сигмоидальной модели.\"\"\"\n",
    "    y_hat = predict_proba(Xb, w)\n",
    "    n = Xb.shape[0]\n",
    "    tmp = (y_hat - y) * y_hat * (1.0 - y_hat)\n",
    "    return (2.0 / n) * (Xb.T @ tmp)\n",
    "\n",
    "g_ce = grad_ce(Xb_train, y_train, w0)\n",
    "g_ls = grad_ls(Xb_train, y_train, w0)\n",
    "print(\"grad_ce shape:\", g_ce.shape, \"norm:\",
np.linalg.norm(g_ce))\n",
    "print(\"grad_ls shape:\", g_ls.shape, \"norm:\", np.linalg.norm(g_ls))"
]
},
{
    "cell_type": "markdown",
    "id": "b429a634",
    "metadata": {},
    "source": [
        "## Блок 6. Градиентный спуск\n",
        "\n",
        "Реализуем универсальную процедуру обучения."
    ]
},
{
    "cell_type": "code",
    "execution_count": null,
    "id": "dc094b24",
    "metadata": {},
    "outputs": [],
    "source": [
        "def gradient_descent(\n",
        "    Xb: np.ndarray,\n",
        "    y: np.ndarray,\n",
        "    loss_fn,\n",
        "    grad_fn,\n",
        "    lr: float = 0.1,\n",
        "    n_iters: int = 2000,\n",
        "    lr_decay: float = 0.0,\n",
        "    verbose_every: int = 200\n",
    ]

```

```

    "):\n",
    "    \\\nУниверсальный градиентный спуск для обучения вектора
w.\\\n",
    "    w = np.zeros(Xb.shape[1])\n",
    "    losses = []\n",
    "\n",
    "    for t in range(1, n_iters + 1):\n",
    "        lr_t = lr / (1.0 + lr_decay * t) if lr_decay > 0 else lr\n",
    "\n",
    "        g = grad_fn(Xb, y, w)\n",
    "        w = w - lr_t * g\n",
    "\n",
    "        if t % 10 == 0 or t == 1:\n",
    "            losses.append(loss_fn(Xb, y, w))\n",
    "\n",
    "        if verbose_every and (t % verbose_every == 0 or t == 1):\n",
    "            print(f"iter={t:5d}  lr={lr_t:.5f}  loss={loss_fn(Xb, y,
w):.6f}")\n",
    "\n",
    "    return w, np.array(losses)"
]
},
{
    "cell_type": "markdown",
    "id": "72412079",
    "metadata": {},
    "source": [
        "## Блок 7. Обучение: CE и LS"
    ]
},
{
    "cell_type": "code",
    "execution_count": null,
    "id": "518beb35",
    "metadata": {},
    "outputs": [],
    "source": [
        "Xb_test = add_bias(X_test_sc)\n",
        "\n",
        "# 1) Обучение с Cross-Entropy\n",
        "w_ce, hist_ce = gradient_descent(\n",
        "    Xb_train, y_train,\n",
        "    loss_fn=loss_ce,\n",
        "    grad_fn=grad_ce,\n",
        "    lr=0.2,\n",
        "    n_iters=3000,\n",
        "    lr_decay=1e-4,\n",
        "    verbose_every=500\n",
        ")\n",
        "\n"
    ]
}

```

```

    "# 2) Обучение с Least Squares\n",
    "w_ls, hist_ls = gradient_descent(\n",
    "    Xb_train, y_train,\n",
    "    loss_fn=loss_ls,\n",
    "    grad_fn=grad_ls,\n",
    "    lr=0.5,\n",
    "    n_iters=3000,\n",
    "    lr_decay=1e-4,\n",
    "    verbose_every=500\n",
    ")\n",
    "\n",
    "print(\"Обучение завершено.\")"
]
},
{
    "cell_type": "code",
    "execution_count": null,
    "id": "c1d1bcbb",
    "metadata": {},
    "outputs": [],
    "source": [
        "# Графики сходимости\n",
        "plt.figure()\n",
        "plt.plot(hist_ce, label=\"Cross-Entropy (CE)\")\n",
        "plt.plot(hist_ls, label=\"Least Squares (LS)\")\n",
        "plt.xlabel(\"Шаги истории (каждые ~10 итераций)\")\n",
        "plt.ylabel(\"Loss\")\n",
        "plt.title(\"Сходимость градиентного спуска: CE vs LS\")\n",
        "plt.legend()\n",
        "plt.show()"
    ]
},
{
    "cell_type": "markdown",
    "id": "4bd9d44f",
    "metadata": {},
    "source": [
        "## Блок 8. Метрики качества: accuracy и confusion matrix"
    ]
},
{
    "cell_type": "code",
    "execution_count": null,
    "id": "5eb008d8",
    "metadata": {},
    "outputs": [],
    "source": [
        "def predict_class(Xb: np.ndarray, w: np.ndarray, threshold: float = 0.5)
-> np.ndarray:\n",
        "    proba = predict_proba(Xb, w)\n",

```



```

    return (proba >= threshold).astype(int)\n",
"\n",
"def accuracy(y_true: np.ndarray, y_pred: np.ndarray) -> float:\n",
"    return float(np.mean(y_true == y_pred))\n",
"\n",
"def confusion_matrix_binary(y_true: np.ndarray, y_pred: np.ndarray):\n",
"    tn = int(np.sum((y_true == 0) & (y_pred == 0)))\n",
"    fp = int(np.sum((y_true == 0) & (y_pred == 1)))\n",
"    fn = int(np.sum((y_true == 1) & (y_pred == 0)))\n",
"    tp = int(np.sum((y_true == 1) & (y_pred == 1)))\n",
"    return tn, fp, fn, tp\n",
"\n",
"def precision_recall(y_true: np.ndarray, y_pred: np.ndarray, eps: float
= 1e-12):\n",
"    tn, fp, fn, tp = confusion_matrix_binary(y_true, y_pred)\n",
"    precision = tp / (tp + fp + eps)\n",
"    recall = tp / (tp + fn + eps)\n",
"    return float(precision), float(recall)\n",
"\n",
"pred_ce = predict_class(Xb_test, w_ce)\n",
"pred_ls = predict_class(Xb_test, w_ls)\n",
"\n",
"print(f"Test accuracy (CE): {accuracy(y_test, pred_ce):.4f}")\n",
"print(f"Test accuracy (LS): {accuracy(y_test, pred_ls):.4f}")\n",
"\n",
"print("\\n\\nConfusion matrix CE (TN, FP, FN, TP):",
confusion_matrix_binary(y_test, pred_ce))\n",
"print("\\nConfusion matrix LS (TN, FP, FN, TP):",
confusion_matrix_binary(y_test, pred_ls))\n",
"\n",
"print("\\n\\nPrecision/Recall CE:", precision_recall(y_test,
pred_ce))\n",
"print("\\nPrecision/Recall LS:", precision_recall(y_test, pred_ls))"
]
},
{
    "cell_type": "markdown",
    "id": "d1e4562f",
    "metadata": {},
    "source": [
        "## Блок 9. Что писать в выводах (шаблон)\n",
        "\n",
        "В отчете обязательно:\n",
        "1) Сравните поведение CE и LS по графикам loss. \n",
        "2) Сравните качество (accuracy + confusion matrix). \n",
        "3) Объясните, почему изменение порога классификации меняет FP/FN. \n",
        "4) Сделайте вывод: какая потеря предпочтительнее и почему (на основе
ваших экспериментов)."
    ]
},

```

```
{
  "cell_type": "markdown",
  "id": "b8c75bc3",
  "metadata": {},
  "source": [
    "# Индивидуальные задания (15 вариантов)\n",
    "\n",
    "> Для каждого варианта: **2 исследовательских задания + 2 контрольных вопроса**.\n",
    "\n",
    "> Выполнение оформляется в отчете: настройки → результаты → выводы.\n",
    "\n",
    "## Вариант 1\n",
    "**Исследовательские задания:**\n",
    "1) Для CE сравнить `lr`: 0.05, 0.1, 0.2 (графики loss).\n",
    "2) Для CE сравнить `threshold`: 0.3, 0.5, 0.7 (confusion matrix).\n",
    "\n",
    "**Контрольные вопросы:**\n",
    "1) Почему порог влияет на FP и FN?\n",
    "2) Почему стандартизация важна для градиентного спуска?\n",
    "\n",
    "## Вариант 2\n",
    "**Исследовательские задания:**\n",
    "1) Для LS сравнить `lr`: 0.1, 0.3, 0.5.\n",
    "2) Для LS сравнить `lr_decay`: 0, 1e-4, 1e-3.\n",
    "\n",
    "**Контрольные вопросы:**\n",
    "1) Чем CE принципиально лучше согласуется с вероятностной интерпретацией?\n",
    "2) Что такое насыщение сигмоиды и почему оно «плохо» для обучения?\n",
    "\n",
    "## Вариант 3\n",
    "**Исследовательские задания:**\n",
    "1) Для CE сравнить `n_iters`: 500, 1500, 3000.\n",
    "2) Построить зависимость accuracy от threshold 0.1...0.9 (шаг 0.1).\n",
    "\n",
    "**Контрольные вопросы:**\n",
    "1) Почему увеличение итераций не всегда улучшает качество на тесте?\n",
    "2) Как понять по loss, что обучение сошлось?\n",
    "\n",
    "## Вариант 4\n",
    "**Исследовательские задания:**\n",
    "1) Удалить стандартизацию и обучить CE при `lr=0.2` (описать, что происходит).\n",
    "2) Построить гистограммы вероятностей `predict_proba` отдельно для классов 0 и 1.\n",
    "\n",
    "**Контрольные вопросы:**\n",
    "1) Почему масштаб признаков меняет геометрию поверхности потерь?
```

```

    "2) Что означает вероятность модели и чем она отличается от
«уверенности»?\\n",
    "\\n",
    "## Вариант 5\\n",
    "**Исследовательские задания:**\\n",
    "1) Провести обучение CE для трех разных `random_state` разбиения и
сравнить метрики. \\n",
    "2) Изменить `test_size`: 0.2 и 0.4 и сравнить качество.\\n",
    "\\n",
    "**Контрольные вопросы:**\\n",
    "1) Зачем фиксировать seed при сравнении экспериментов? \\n",
    "2) Почему важно использовать stratify?\\n",
    "\\n",
    "## Варианты 6-15\\n",
    "Сформируйте свой вариант по шаблону 1-5: \\n",
    "- выберите один фактор (lr, lr_decay, n_iters, threshold, разбиение),
\\n",
    "- сделайте 2 эксперимента, \\n",
    "- ответьте на 2 контрольных вопроса (сформулируйте их сами, по теме
варианта).\"
]
},
{
    \"cell_type\": \"markdown\",
    \"id\": \"896bb2b7\",
    \"metadata\": {},
    \"source\": [
        \"# Чек-лист отчета студента\\n\",
        \"\\n\",
        \"1. ФИО, группа, вариант. \\n\",
        \"2. Цель работы (2-4 предложения). \\n\",
        \"3. Описание данных и предобработки (в т.ч. стандартизация). \\n\",
        \"4. Формулы CE и LS + объяснение смысла. \\n\",
        \"5. Формула градиентного спуска и гиперпараметры. \\n\",
        \"6. Эксперименты по варианту: графики, метрики, выводы. \\n\",
        \"7. Ответы на контрольные вопросы. \\n\",
        \"8. Итоговые выводы (не менее 10-12 предложений).\"
    ]
}
],
\"metadata\": {
    \"language_info\": {
        \"name\": \"python\"
    }
},
\"nbformat\": 4,
\"nbformat_minor\": 5
}

```